



deeplearning.ai

Case Studies

Why look at
case studies?

Outline

Classic networks:

- LeNet-5 ←
- AlexNet ←
- VGG ←

ResNet (152)

Inception

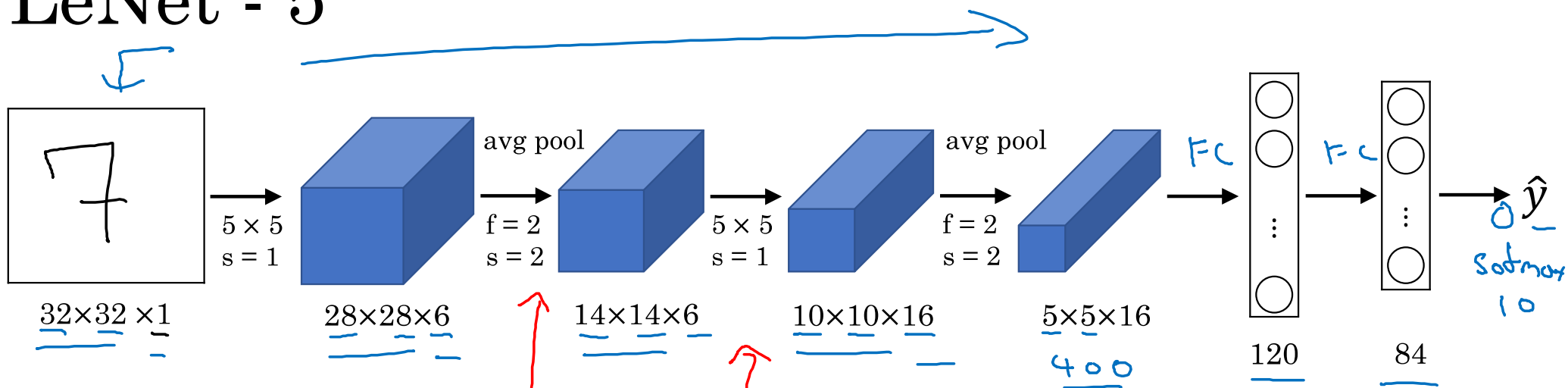


deeplearning.ai

Case Studies

Classic networks

LeNet - 5



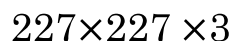
60K parameters.

$n_H, n_w \downarrow \quad n_c \uparrow$

conv pool conv pool fc fc output

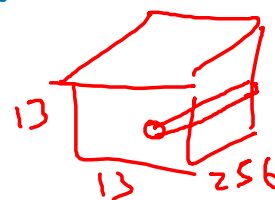
Advanced: sigmoid/tanh ReLU

II III



- 9216

nom parameters



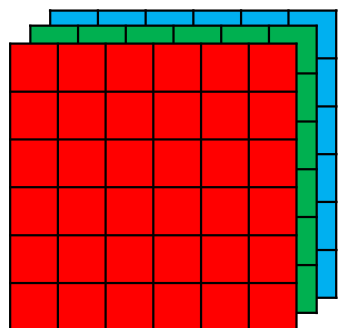
Andrew Ng

VGG - 16

VGG-19

CONV = 3x3 filter, s = 1, same

MAX-POOL = 2x2, s = 2



224x224x3

[CONV 64]
x2

224x224x64

POOL

112x112x64

[CONV 128]
x2

112x112x128

POOL

56x56x128

[CONV 256]
x3

56x56x256

POOL

28x28x256

[CONV 512]
x3

28x28x512

POOL

14x14x512

[CONV 512]
x3

14x14x512

POOL

7x7x512

FC
4096

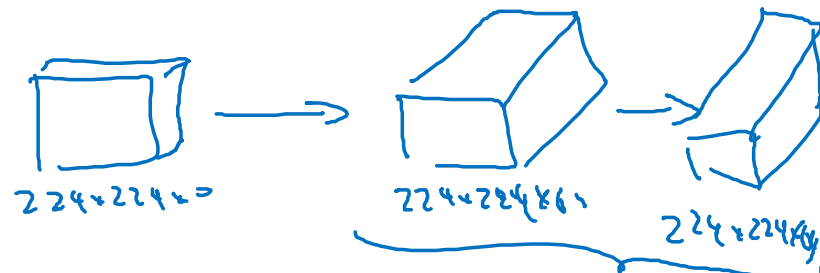
FC
4096

Softmax
1000

$n_H, n_W \downarrow$

$n_C \uparrow$

~138M



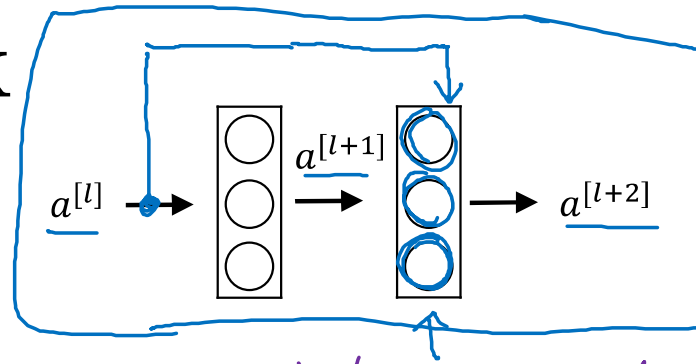


deeplearning.ai

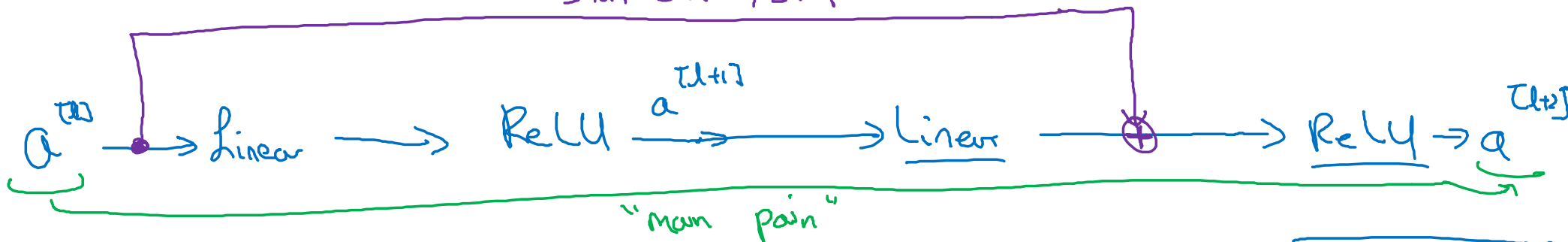
Case Studies

Residual Networks (ResNets)

Residual block



"short cut" / skip connection



$$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]}$$

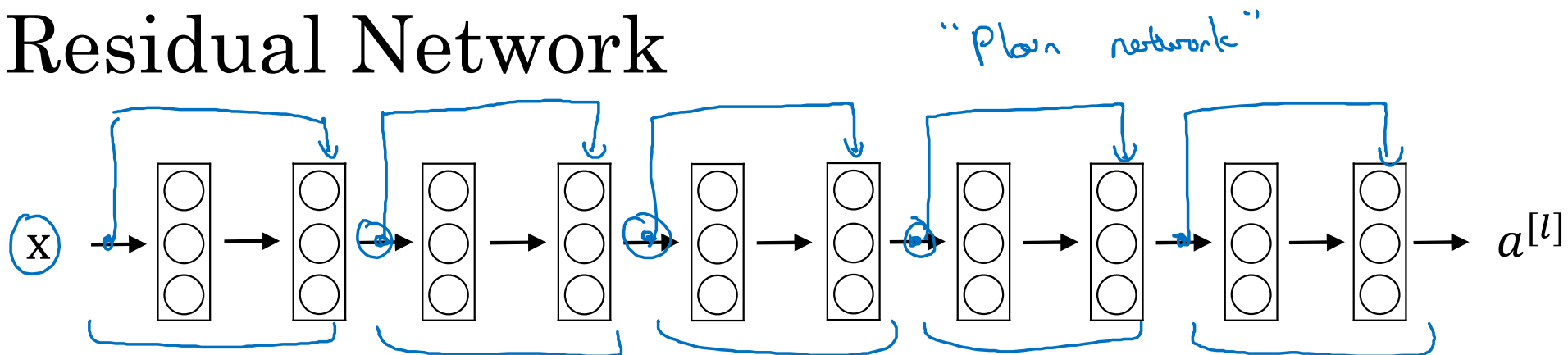
$$a^{[l+1]} = g(z^{[l+1]})$$

$$z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]}$$

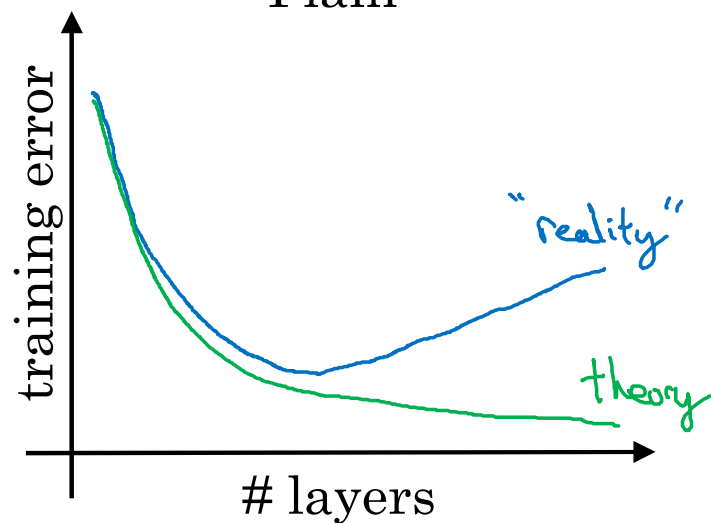
~~$$a^{[l+2]} = g(z^{[l+2]})$$~~

$$a^{[l+2]} = g(z^{[l+2]} + \underbrace{a^{[l]}})$$

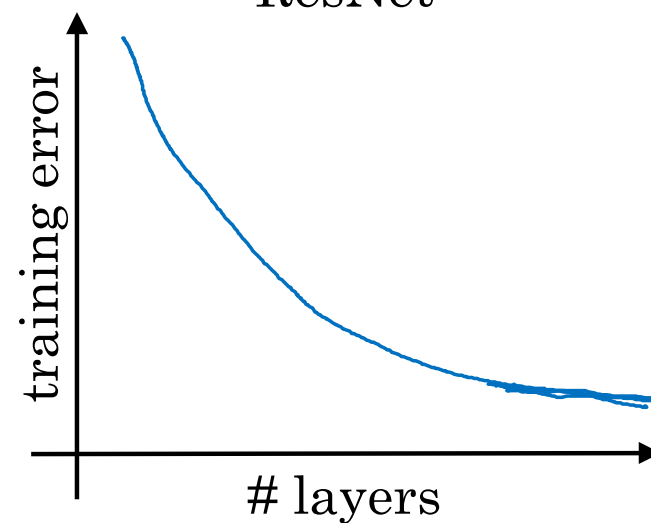
Residual Network



Plain



ResNet



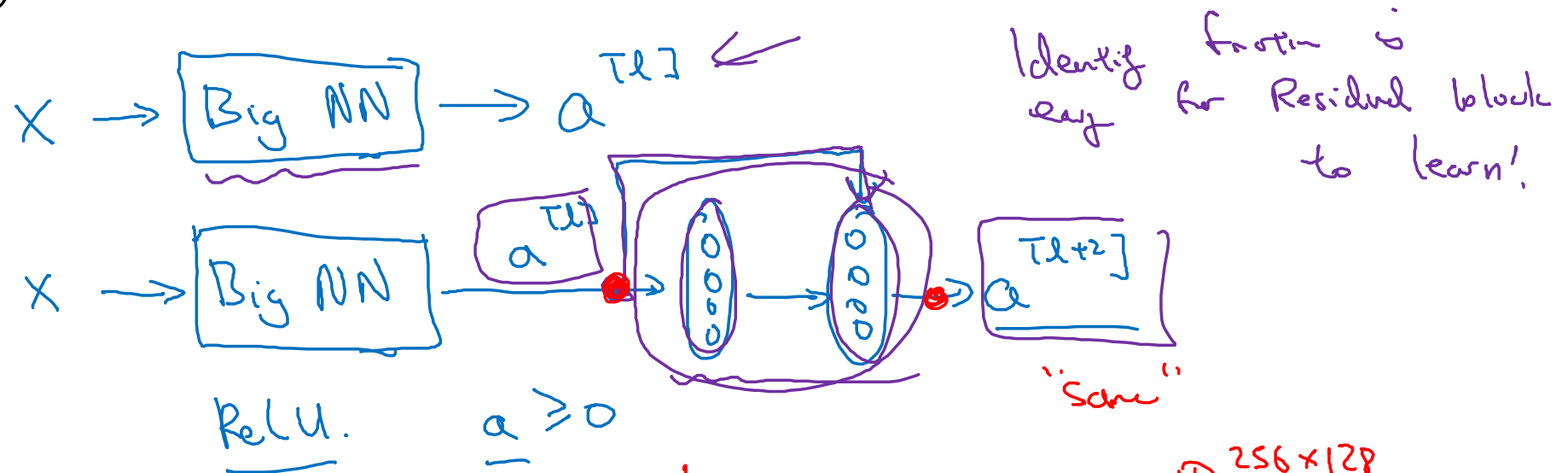


deeplearning.ai

Case Studies

Why ResNets work

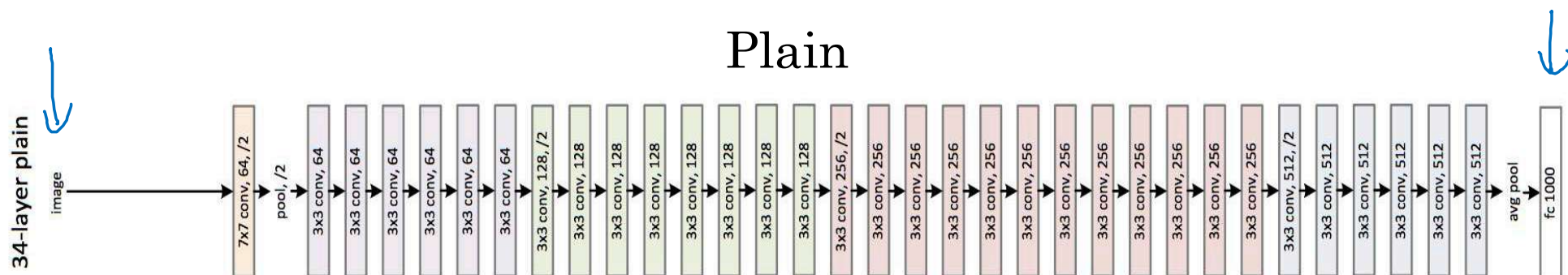
Why do residual networks work?



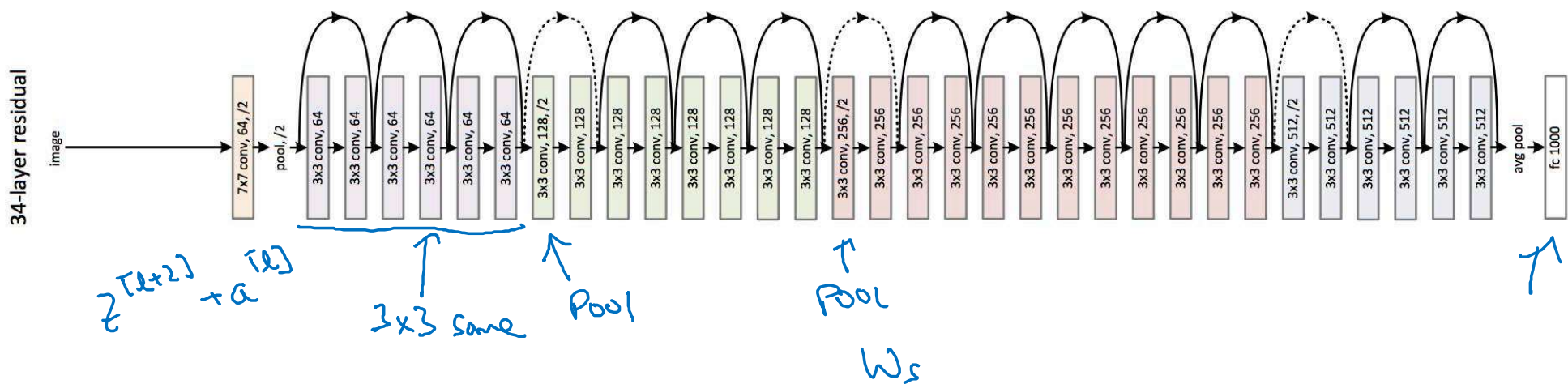
$$\begin{aligned}
 a^{[l+2]} &= g(z^{[l+2]} + a^{[l]}) \\
 &= g(\underbrace{w^{[l+2]}_0}_{\text{IF } w^{[l+2]} = 0} + \underbrace{a^{[l+1]}}_{\text{IF } b^{[l+2]} = 0} + \underbrace{w_s^{[l+2]} a^{[l]}}_{\text{IF } w_s^{[l+2]} = 0}) = g(a^{[l]}) = \underline{a^{[l]}}
 \end{aligned}$$

ResNet

Plain



ResNet



[He et al., 2015. Deep residual networks for image recognition]

Andrew Ng



deeplearning.ai

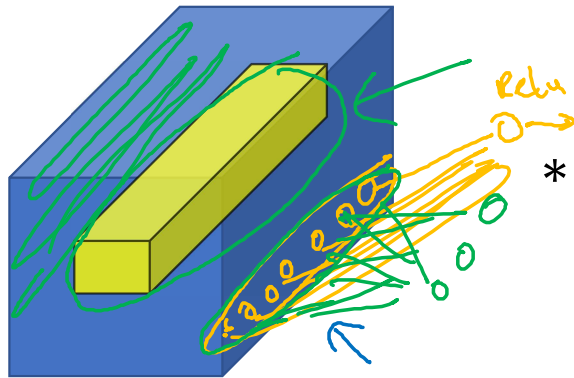
Case Studies

Network in Network
and 1×1 convolutions

Why does a 1×1 convolution do?

1	2	3	6	5	8
3	5	5	1	3	4
2	1	3	4	9	3
4	7	8	5	7	9
1	5	3	7	4	8
5	4	9	8	3	5

$6 \times 6 \times 1$



$6 \times 6 \times 32$

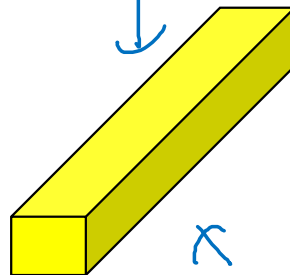
*

2

=

32

→ # filters.
 $n_c^{[L+1]}$



$1 \times 1 \times 32$

=

ReLU

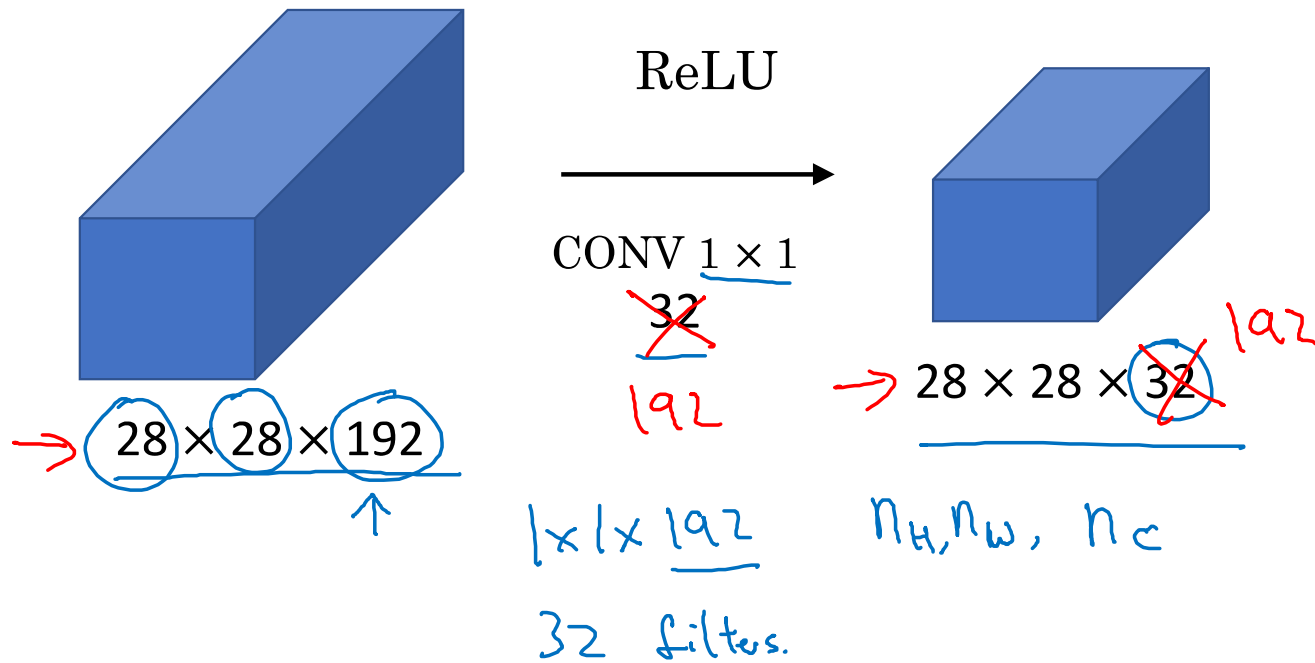
Network in
Network

2	4	6	...		

$6 \times 6 \times \# \text{ filters}$

Andrew Ng

Using 1×1 convolutions



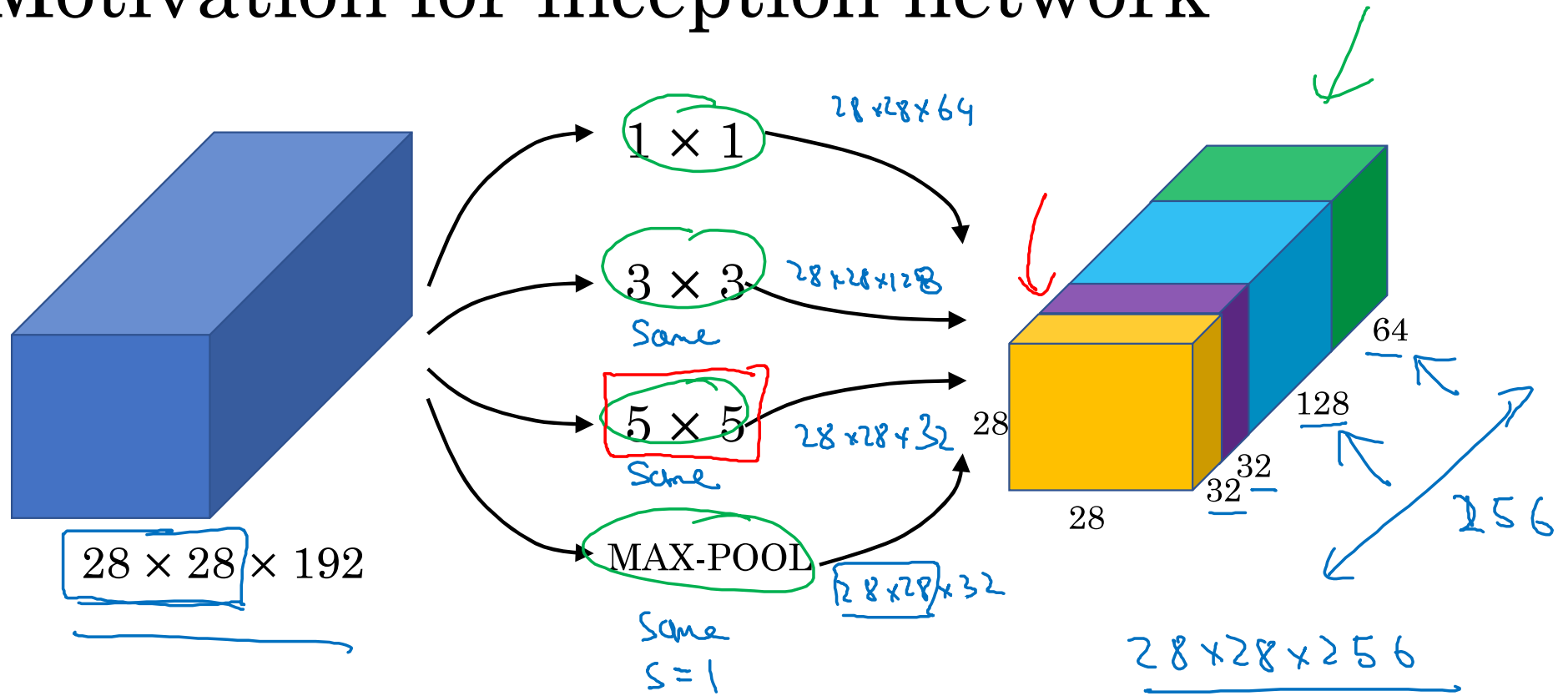


deeplearning.ai

Case Studies

Inception network motivation

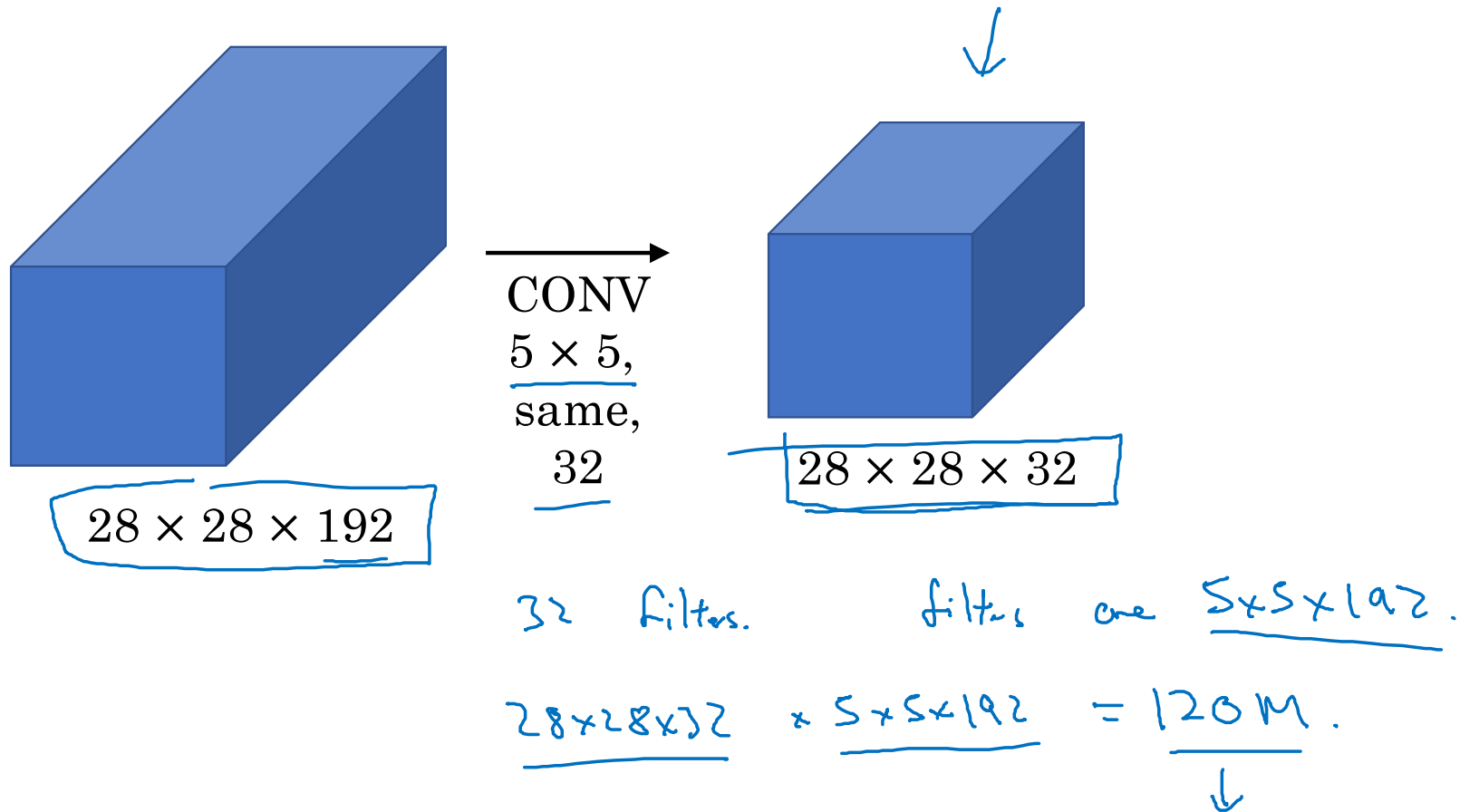
Motivation for inception network



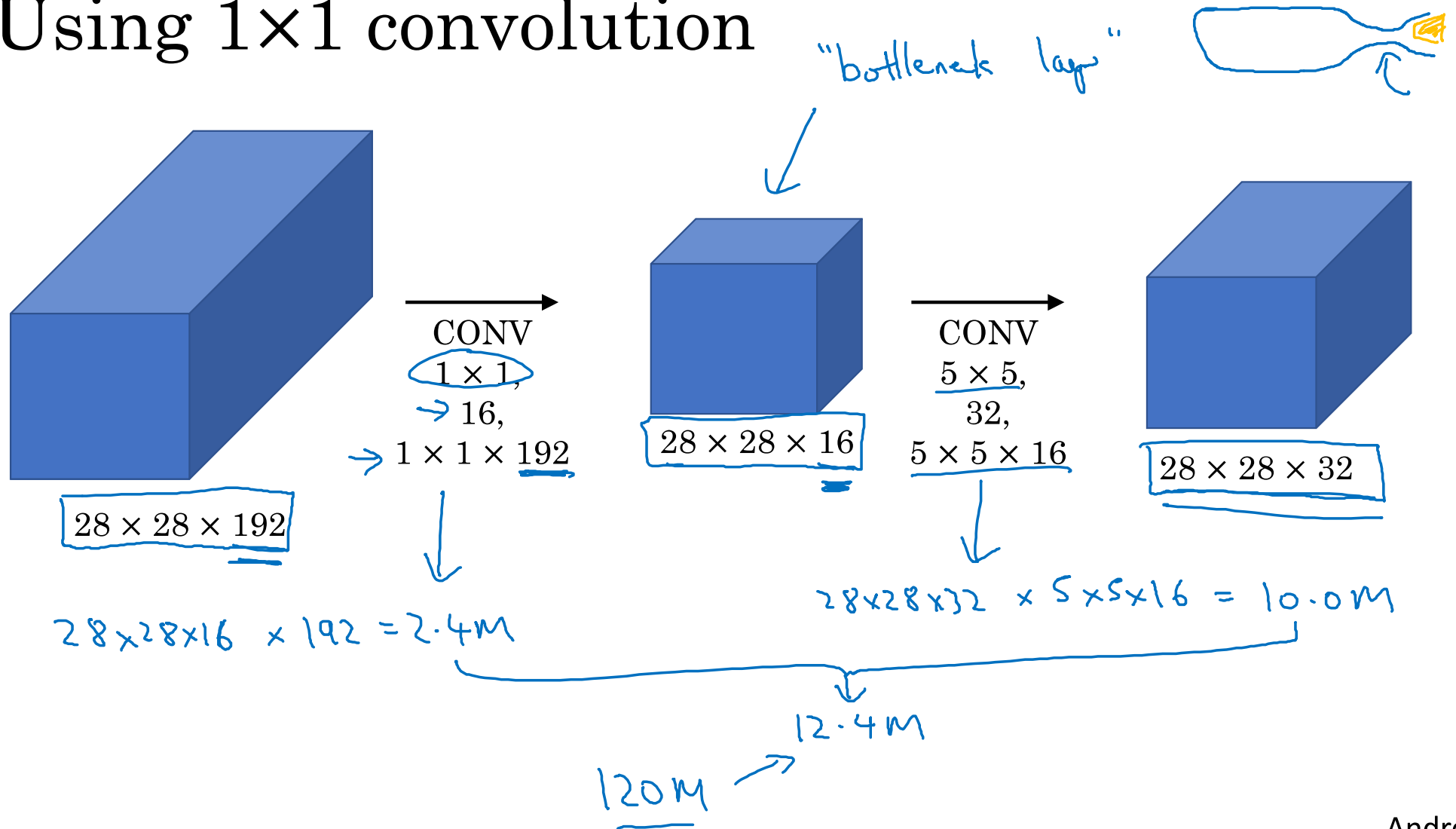
[Szegedy et al. 2014. Going deeper with convolutions]

Andrew Ng

The problem of computational cost



Using 1×1 convolution



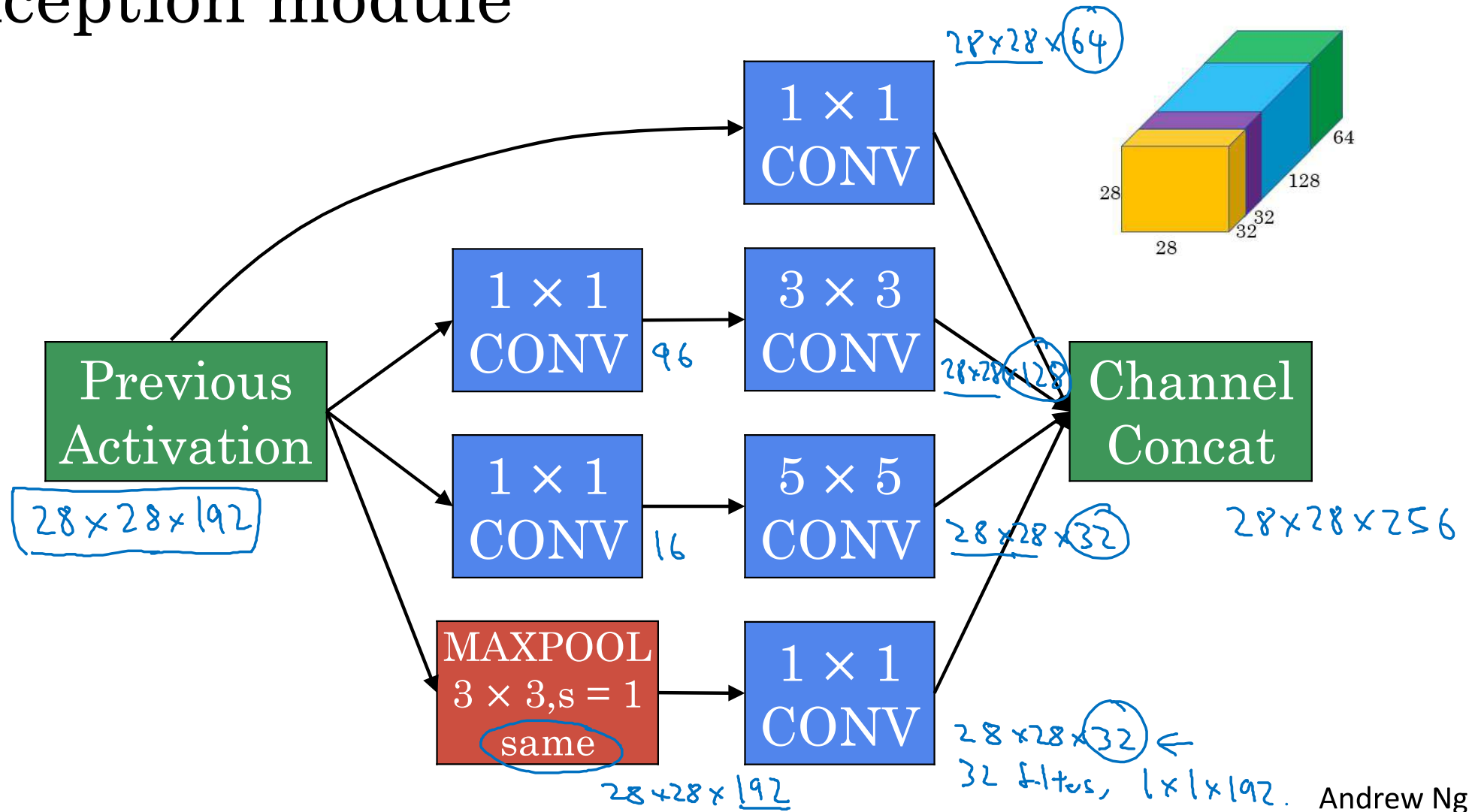


deeplearning.ai

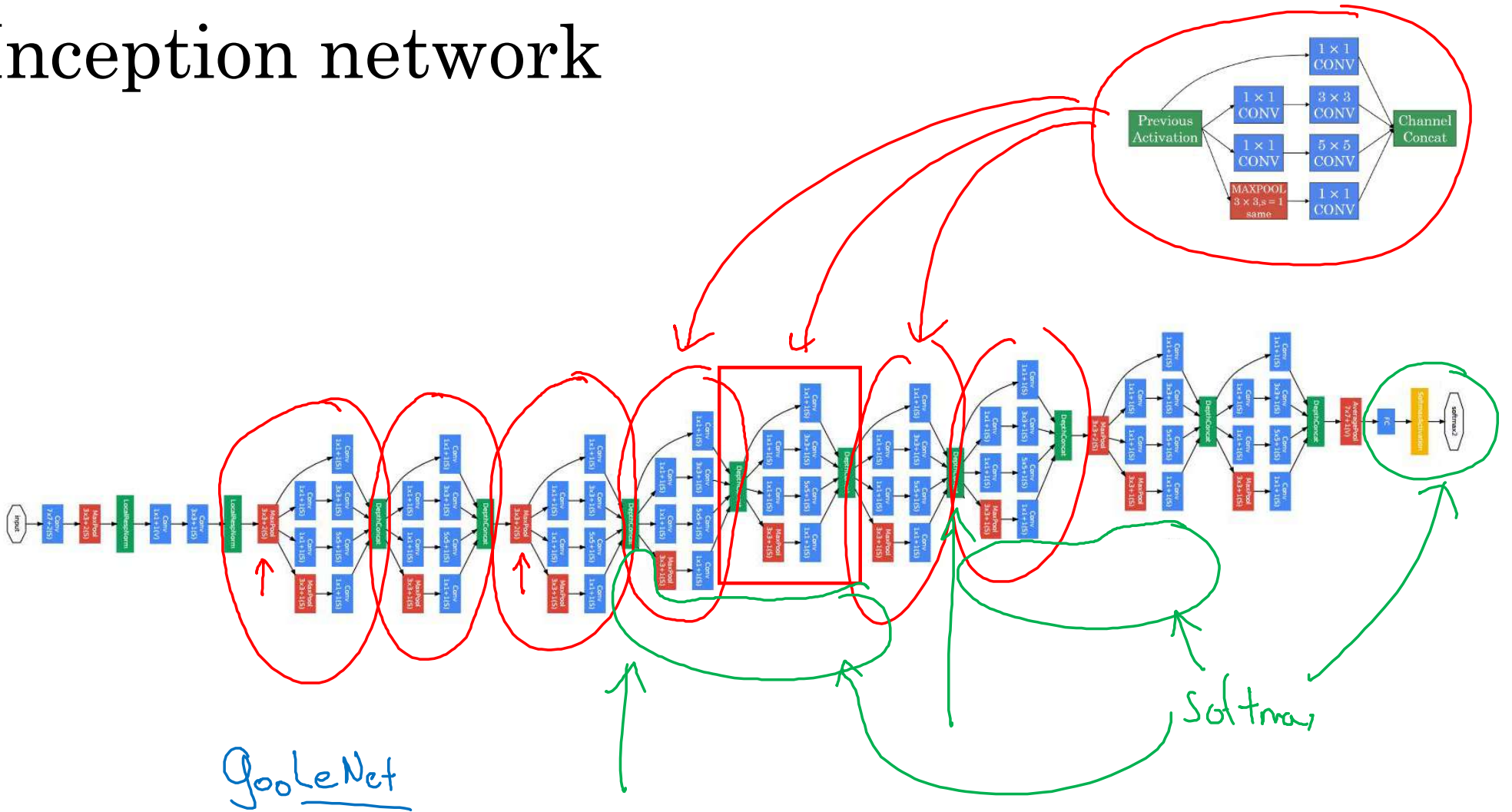
Case Studies

Inception network

Inception module



Inception network



[Szegedy et al., 2014, Going Deeper with Convolutions]

Andrew Ng



<http://knowyourmeme.com/memes/we-need-to-go-deeper> 

Andrew Ng



deeplearning.ai

Convolutional Neural Networks

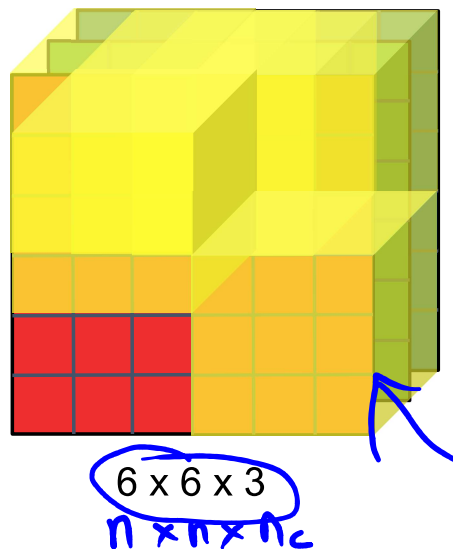
MobileNet

Motivation for MobileNets

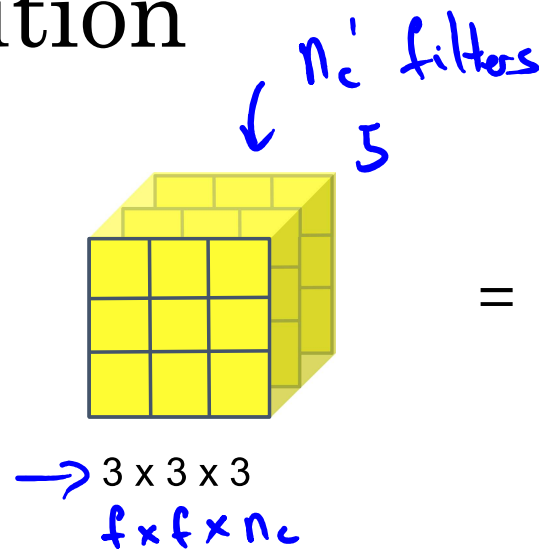
- Low computational cost at deployment
- Useful for mobile and embedded vision applications
- Key idea: Normal vs. depthwise-separable convolutions



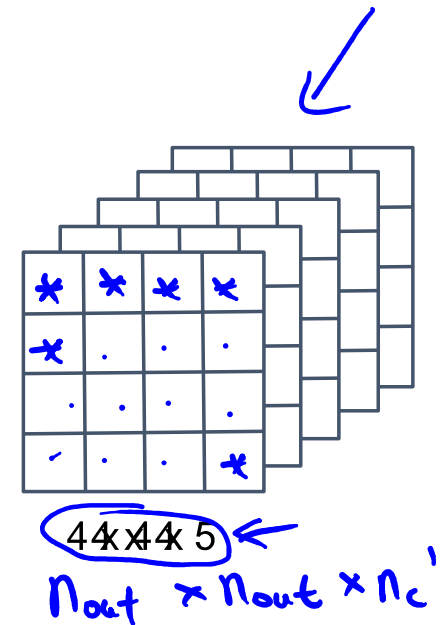
Normal Convolution



*



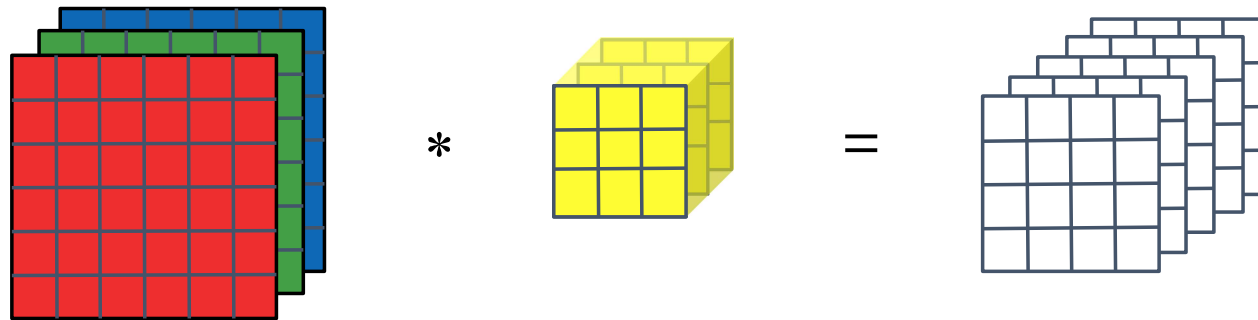
=



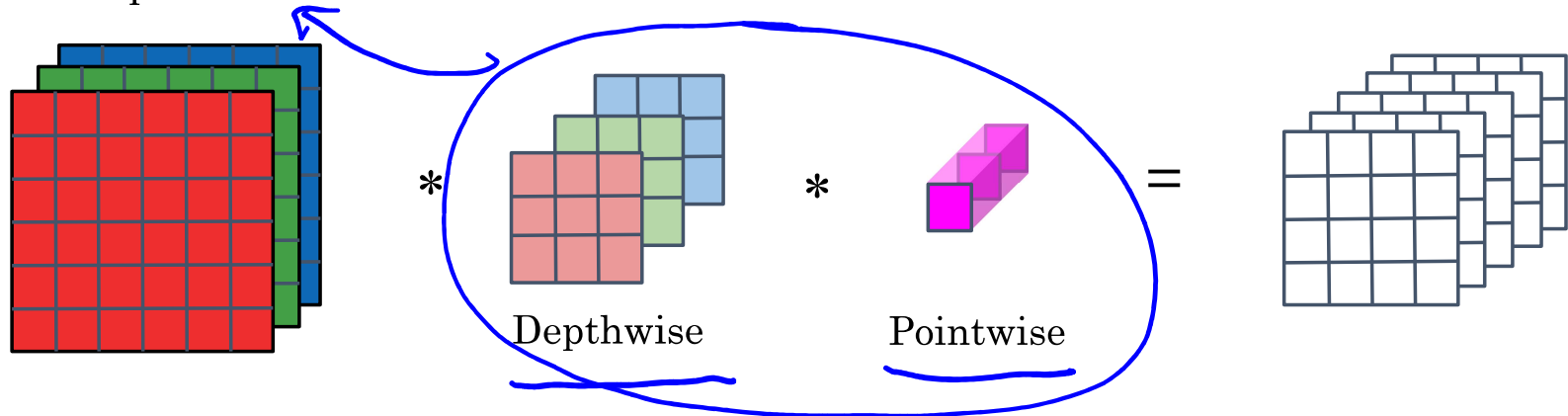
$$\begin{aligned} \text{Computational cost} &= \frac{\text{\#filter params}}{3 \times 3 \times 3} \times \frac{\text{\# filter positions}}{4 \times 4} \times \text{\# of filters} \\ &\rightarrow \underline{2160} = \frac{3 \times 3 \times 3}{\uparrow} \times \frac{4 \times 4}{\uparrow} \times \frac{5}{\uparrow} \end{aligned}$$

Depthwise Separable Convolution

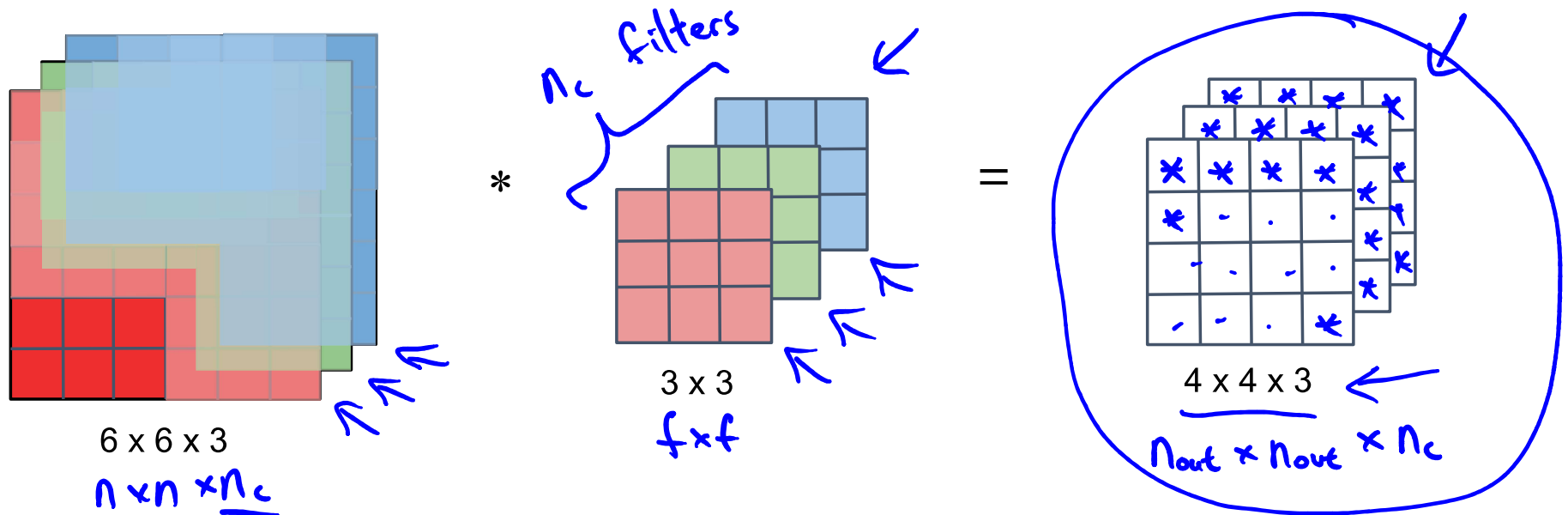
Normal Convolution



Depthwise Separable Convolution



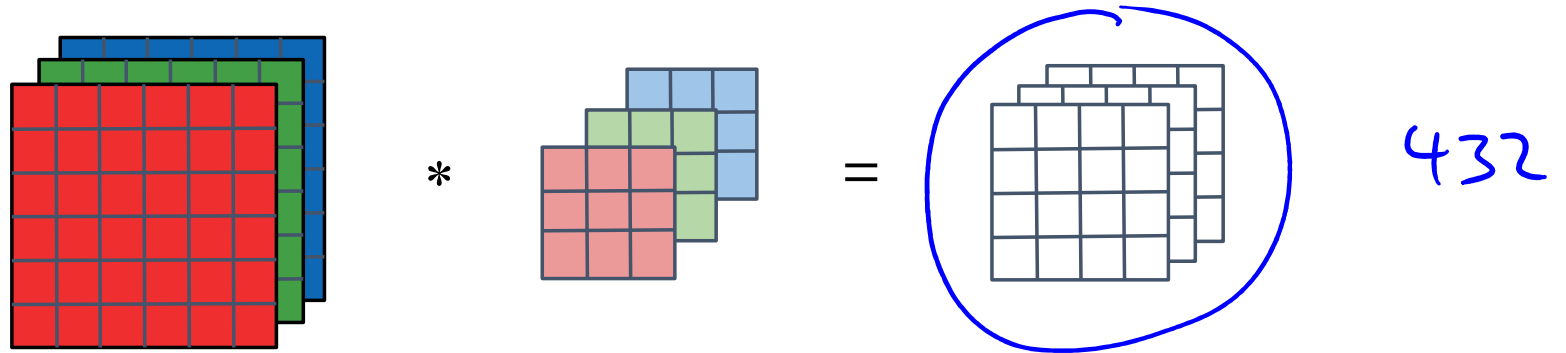
Depthwise Convolution



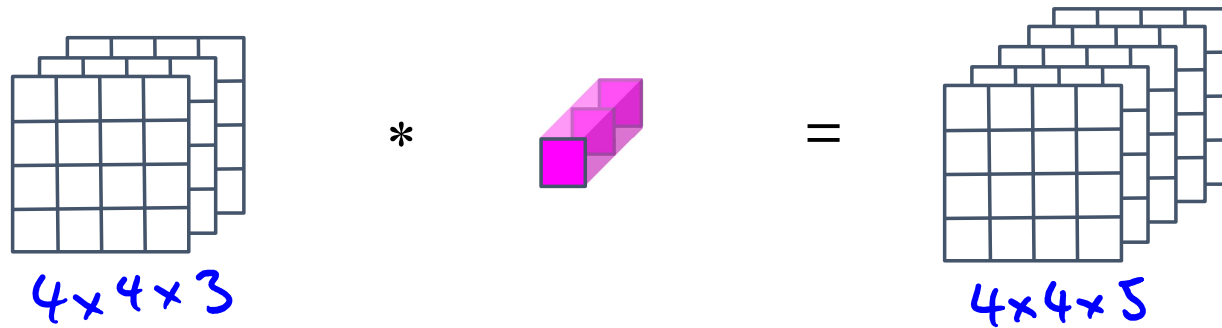
$$\begin{array}{ccccccc}
 \text{Computational cost} & = & \# \text{filter params} & \times & \# \text{filter positions} & \times & \# \text{of filters} \\
 432 & = & \underbrace{3 \times 3} & \times & \underbrace{4 \times 4} & \times & 3
 \end{array}$$

Depthwise Separable Convolution

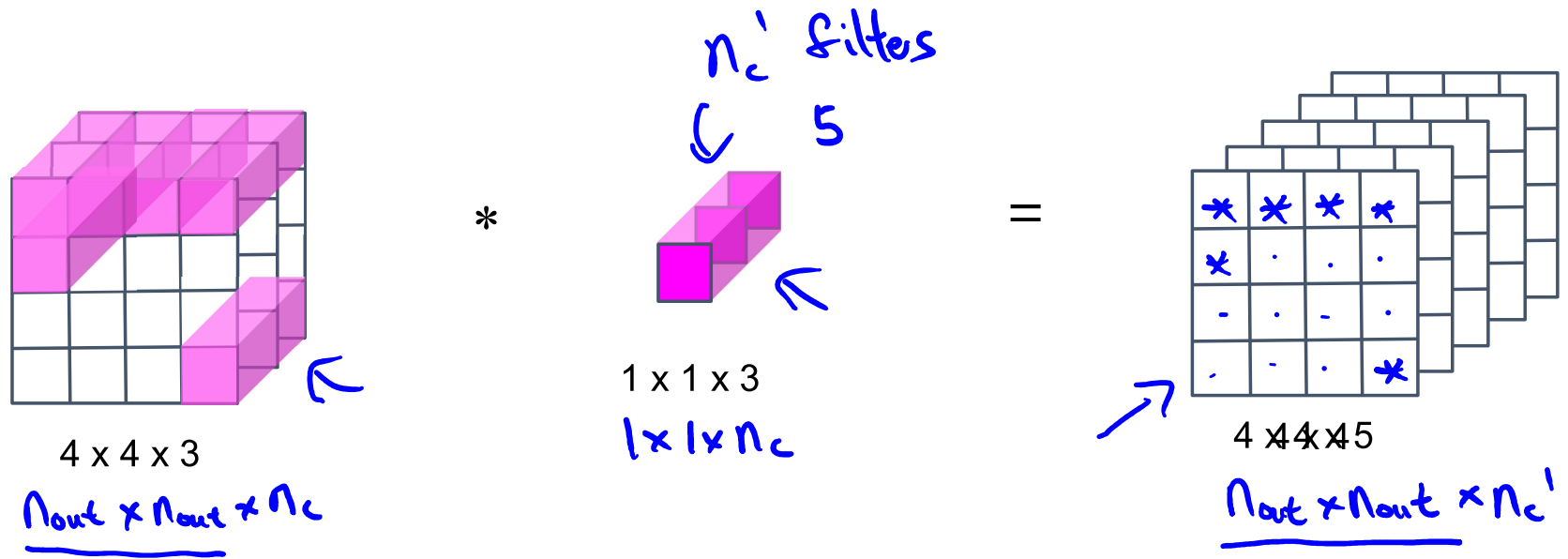
Depthwise Convolution



Pointwise Convolution



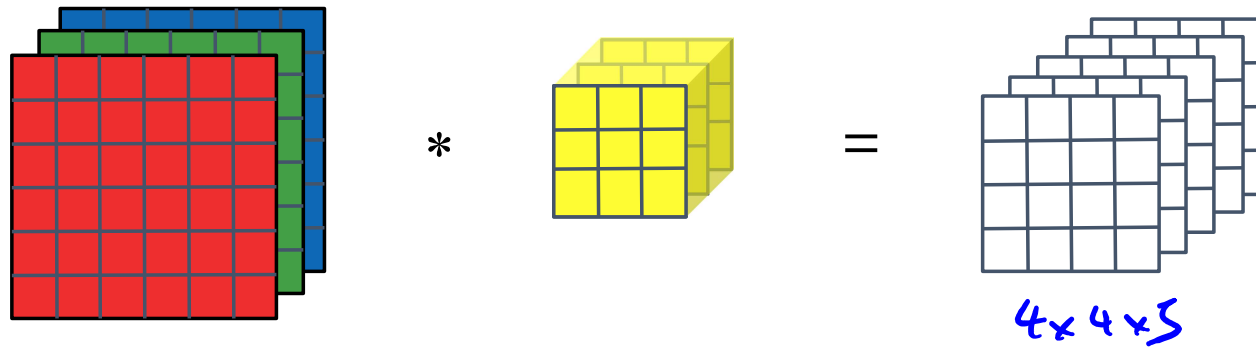
Pointwise Convolution



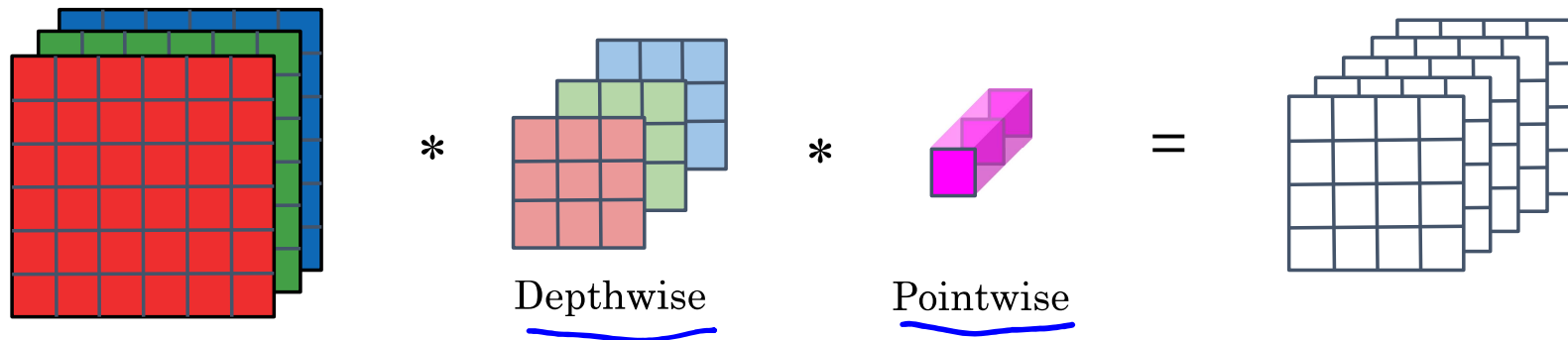
$$\begin{array}{ccccccc}
 \text{Computational cost} & = & \text{\#filter params} & \times & \text{\# filter positions} & \times & \text{\# of filters} \\
 240 & = & 1 \times 1 \times 3 & \times & 4 \times 4 & \times & 5
 \end{array}$$

Depthwise Separable Convolution

Normal Convolution



Depthwise Separable Convolution



Cost Summary

Cost of normal convolution

2160

Cost of depthwise separable convolution

$$\begin{array}{ccc} \text{depthwise} & + & \text{pointwise} \\ 432 & + & 240 = 672 \end{array}$$

$$\frac{672}{2160} = 0.31 \leftarrow$$

$$= \frac{1}{\textcircled{n_c}} + \frac{1}{f^2}$$
$$\frac{1}{5} + \frac{1}{9}$$

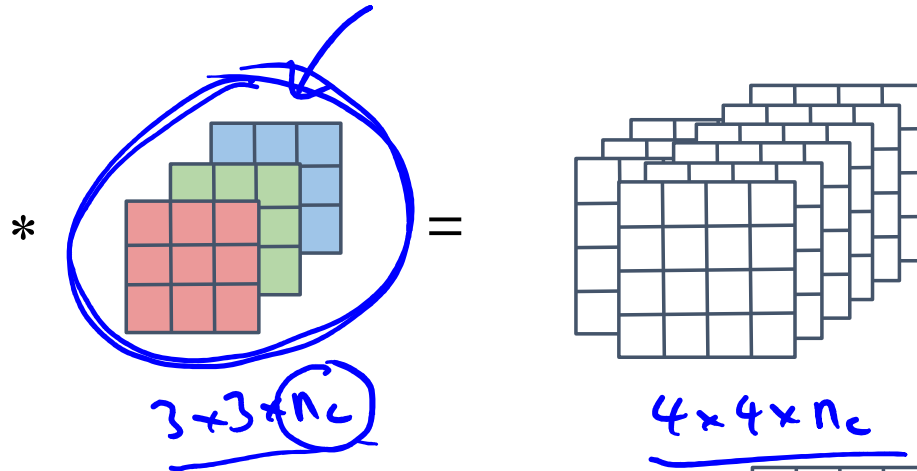
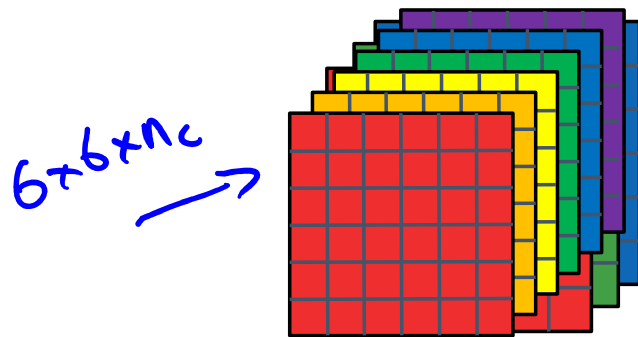
$$= \frac{1}{512} + \frac{1}{3^2}$$

↑ ↑

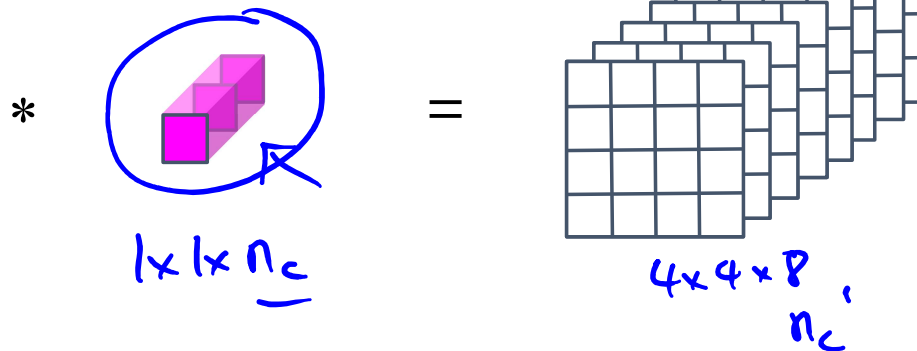
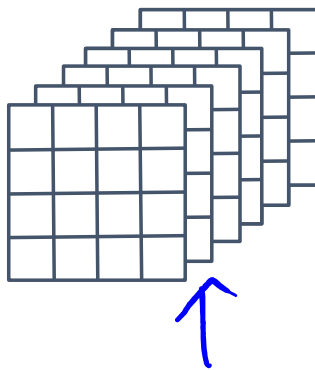
~10 times cheaper

Depthwise Separable Convolution

Depthwise Convolution



Pointwise Convolution





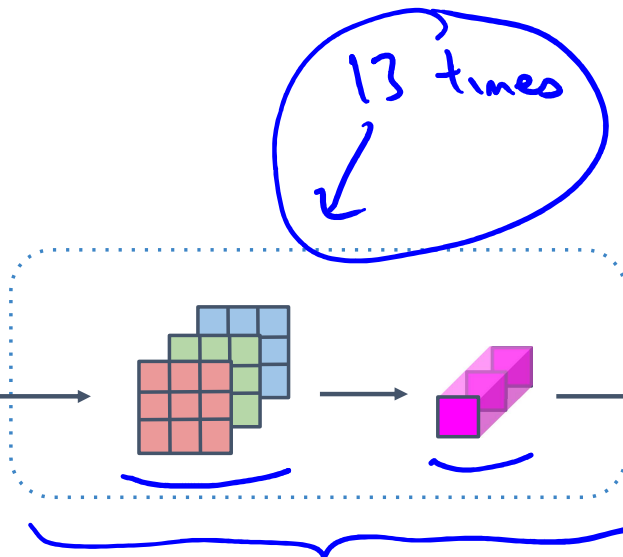
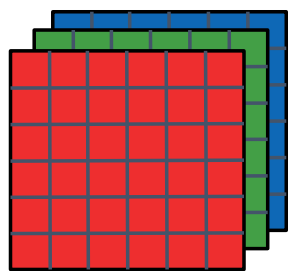
deeplearning.ai

Convolutional Neural Networks

MobileNet Architecture

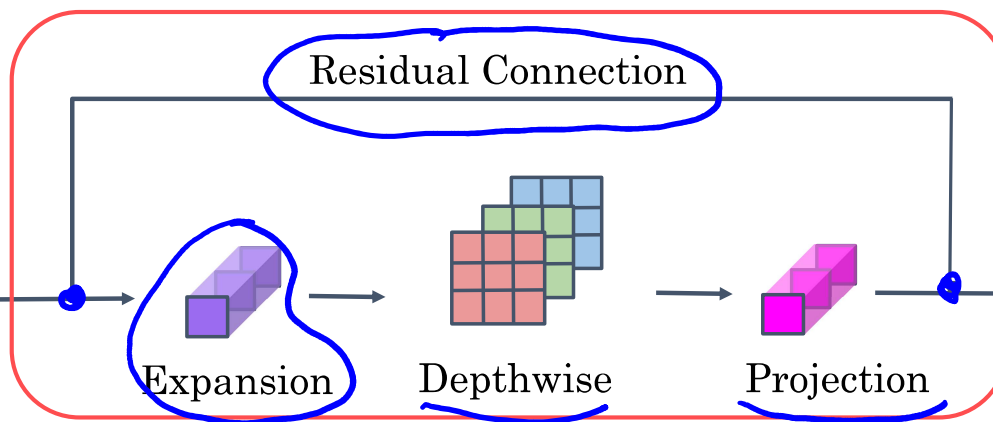
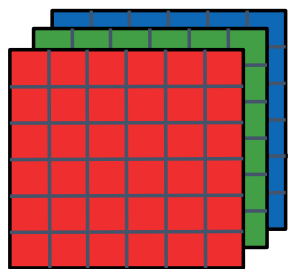
MobileNet

MobileNet v1



Pool, FC, Softmax

MobileNet v2



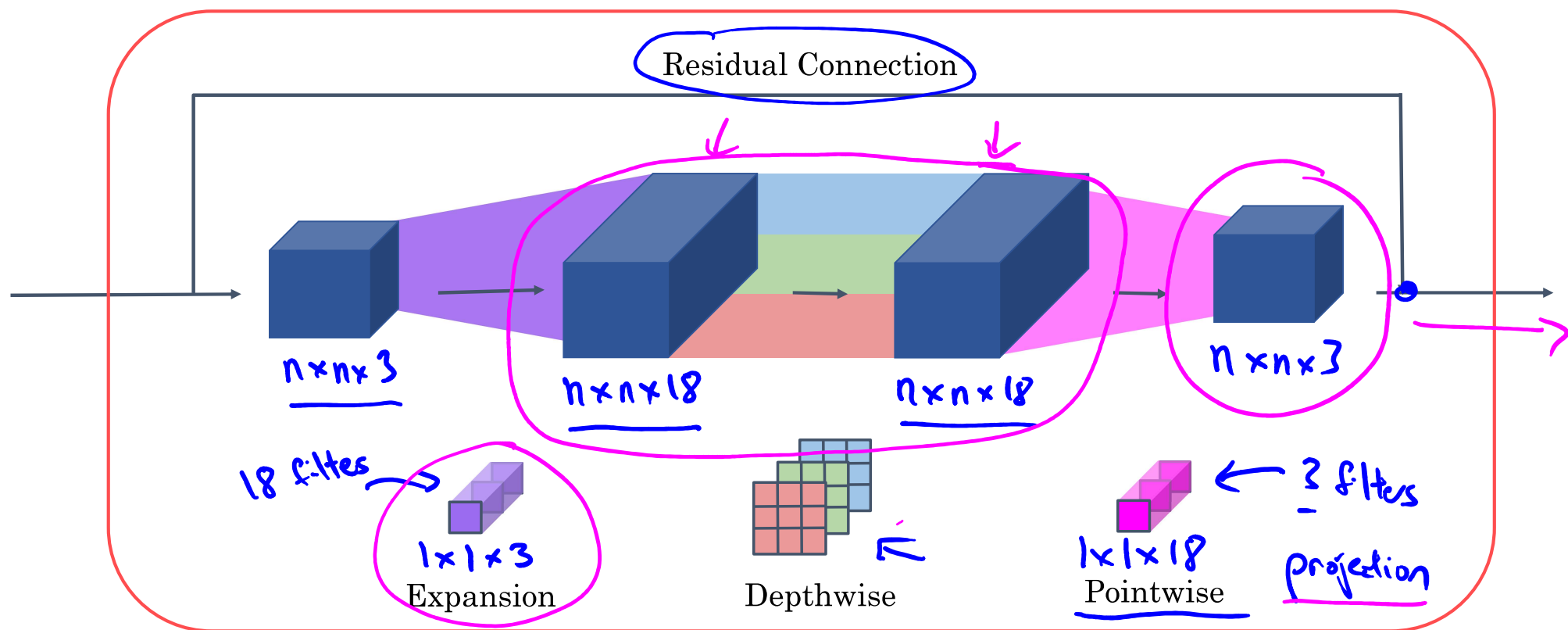
Pool, FC, Softmax

Bottleneck

[Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks]

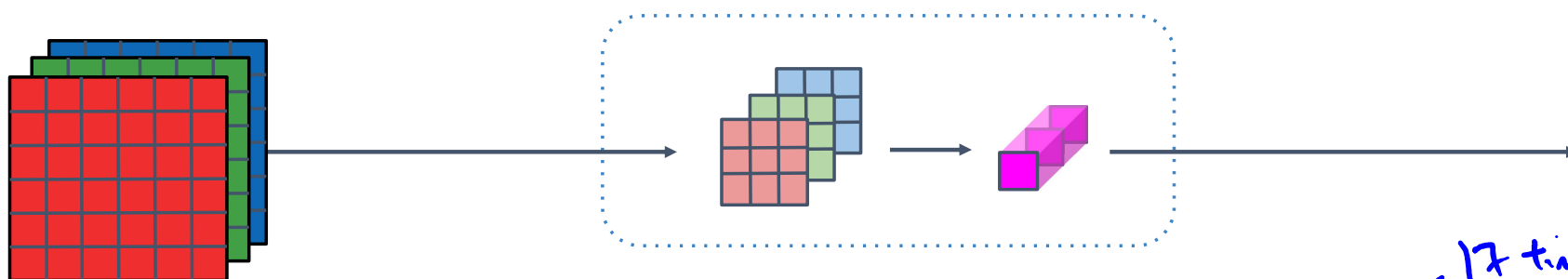
Andrew Ng

MobileNet v2 Bottleneck

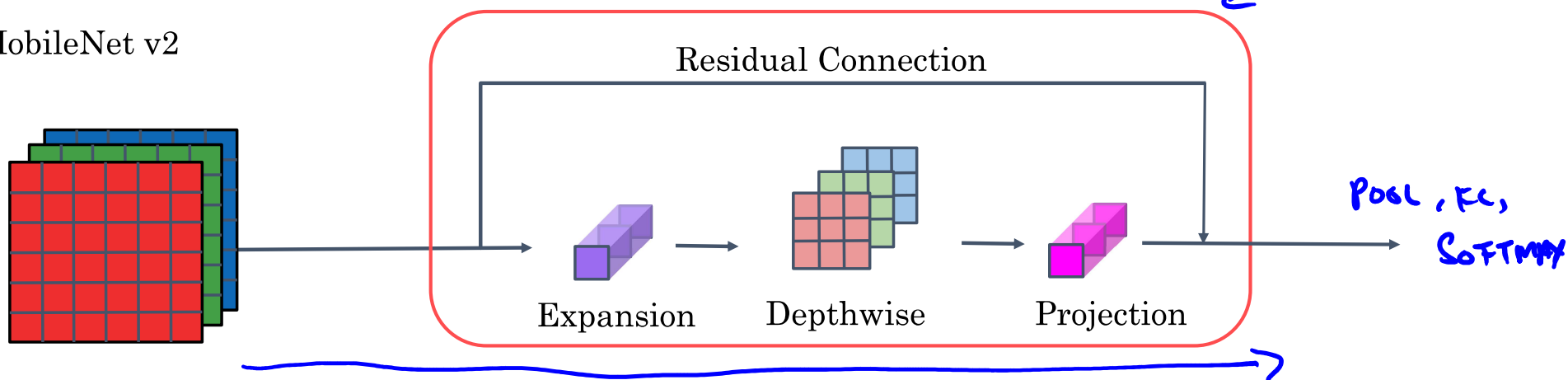


MobileNet

MobileNet v1



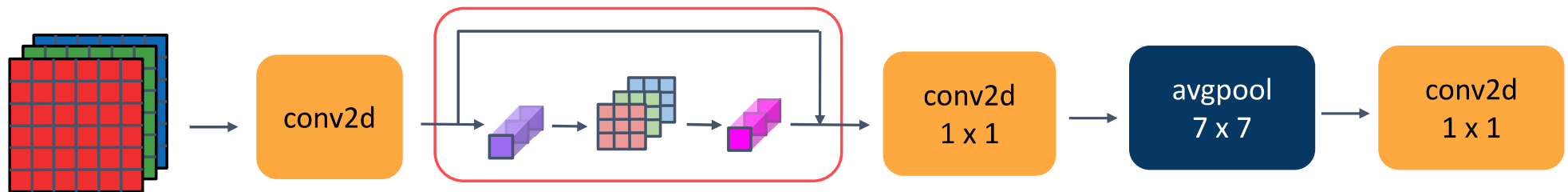
MobileNet v2



[Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks]

Andrew Ng

MobileNet v2 Full Architecture





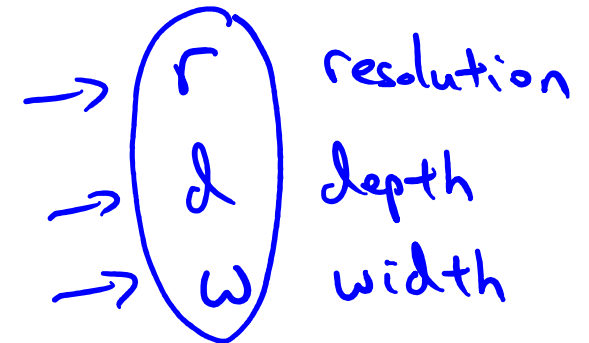
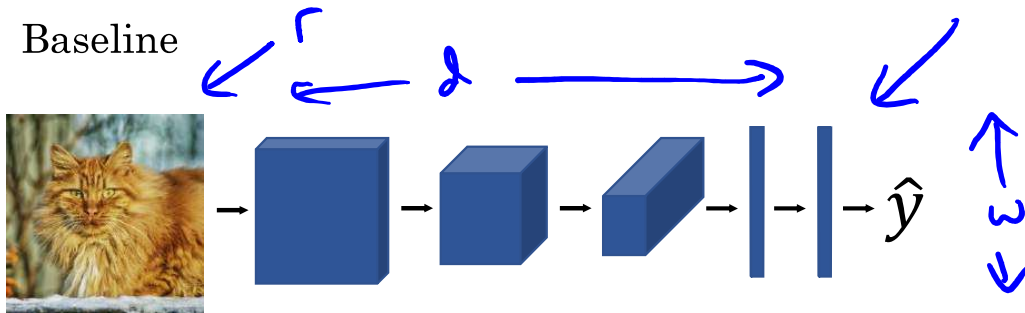
deeplearning.ai

Convolutional Neural Networks

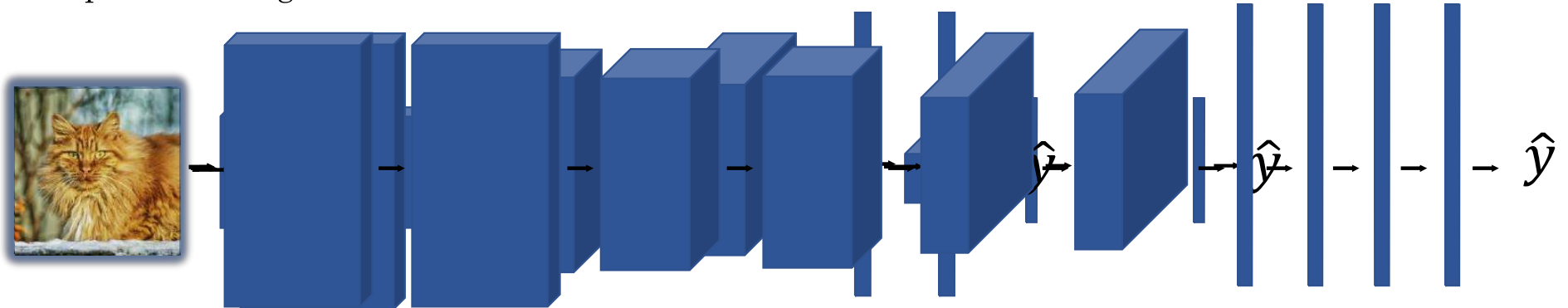
EfficientNet

EfficientNet

Baseline



Width Rescaling



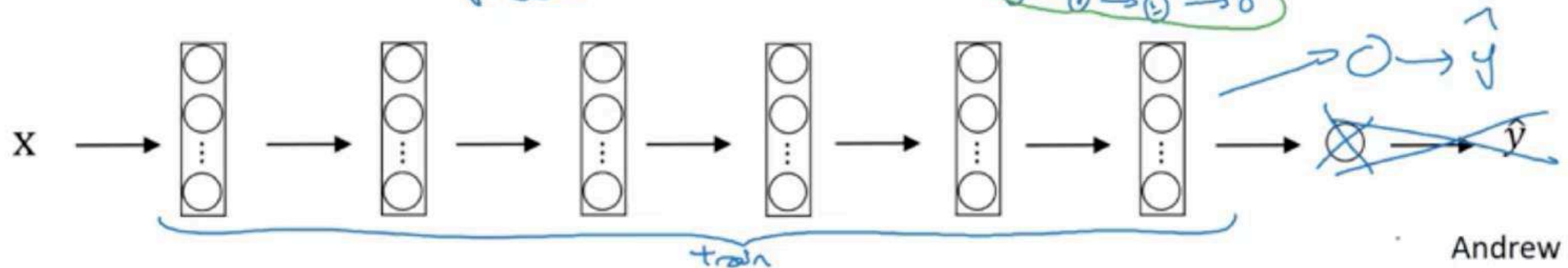
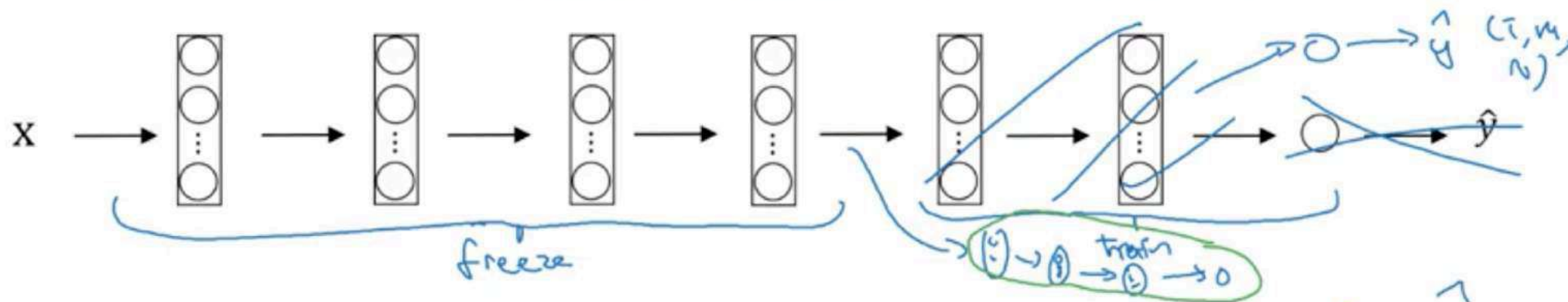
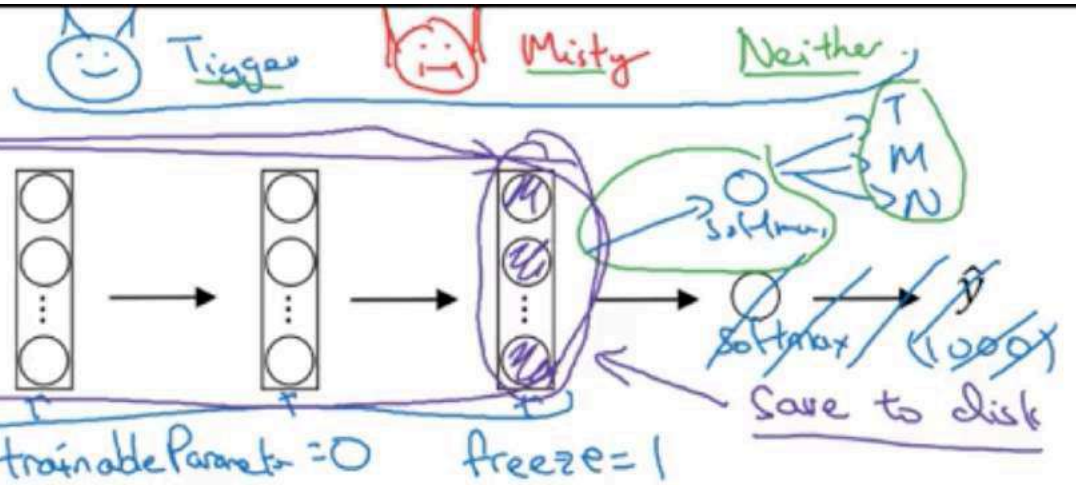


deeplearning.ai

Practical advice for
using ConvNets

Transfer Learning

Transfer Learning





deeplearning.ai

Practical advice for using ConvNets

Data augmentation

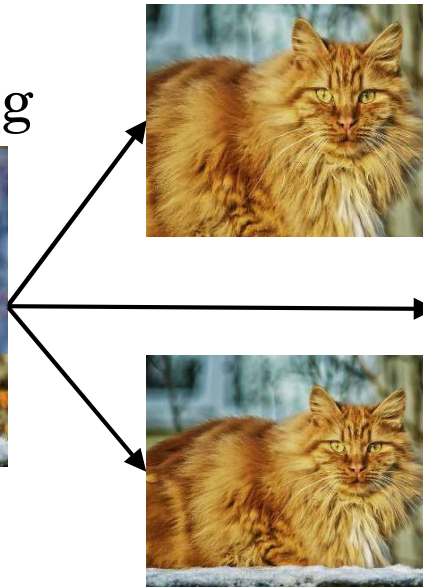
Common augmentation method

Mirroring



y

Random Cropping



Rotation

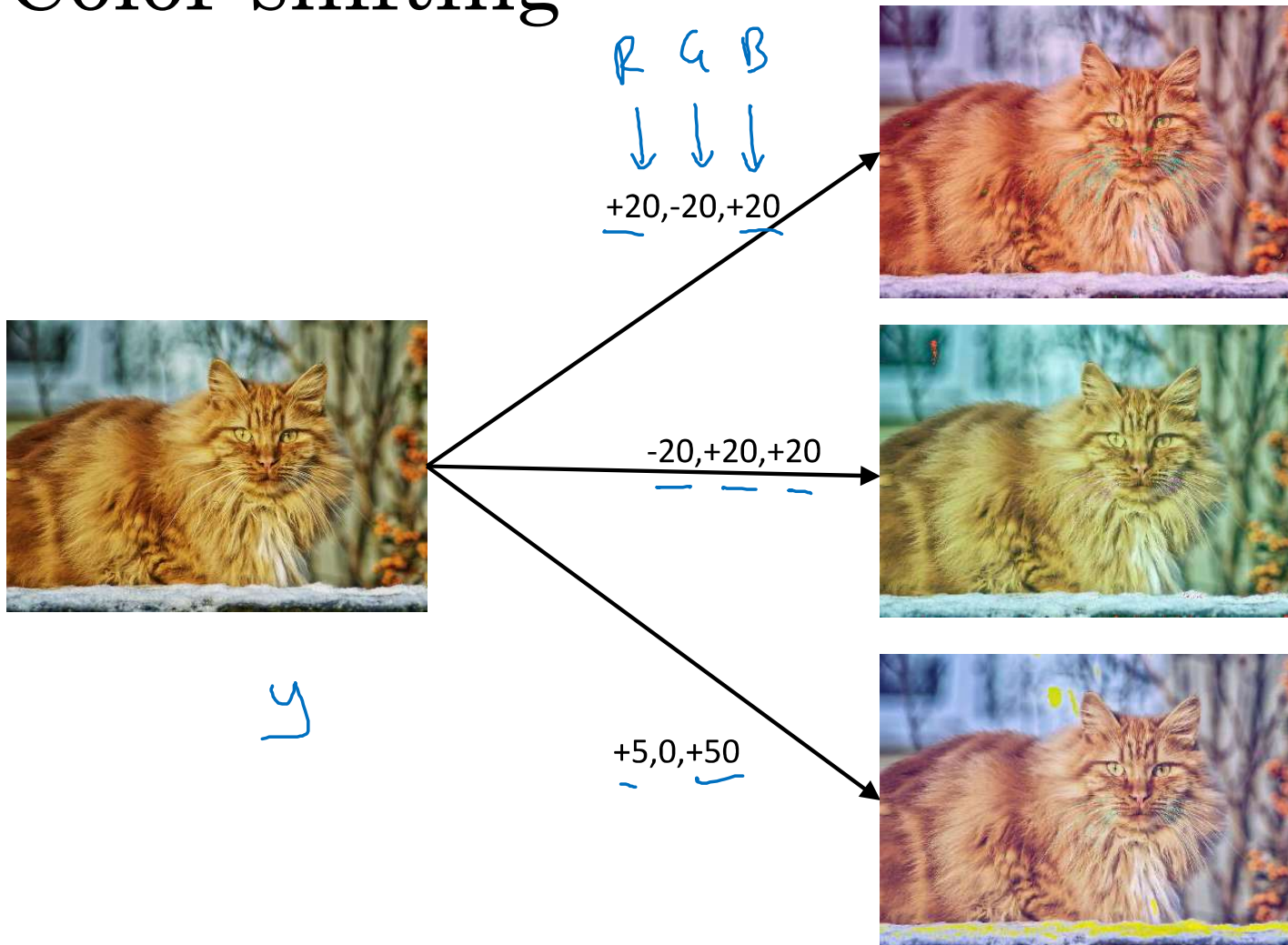
Shearing

Local warping

...



Color shifting



Advanced:

PCA

ml-class.org

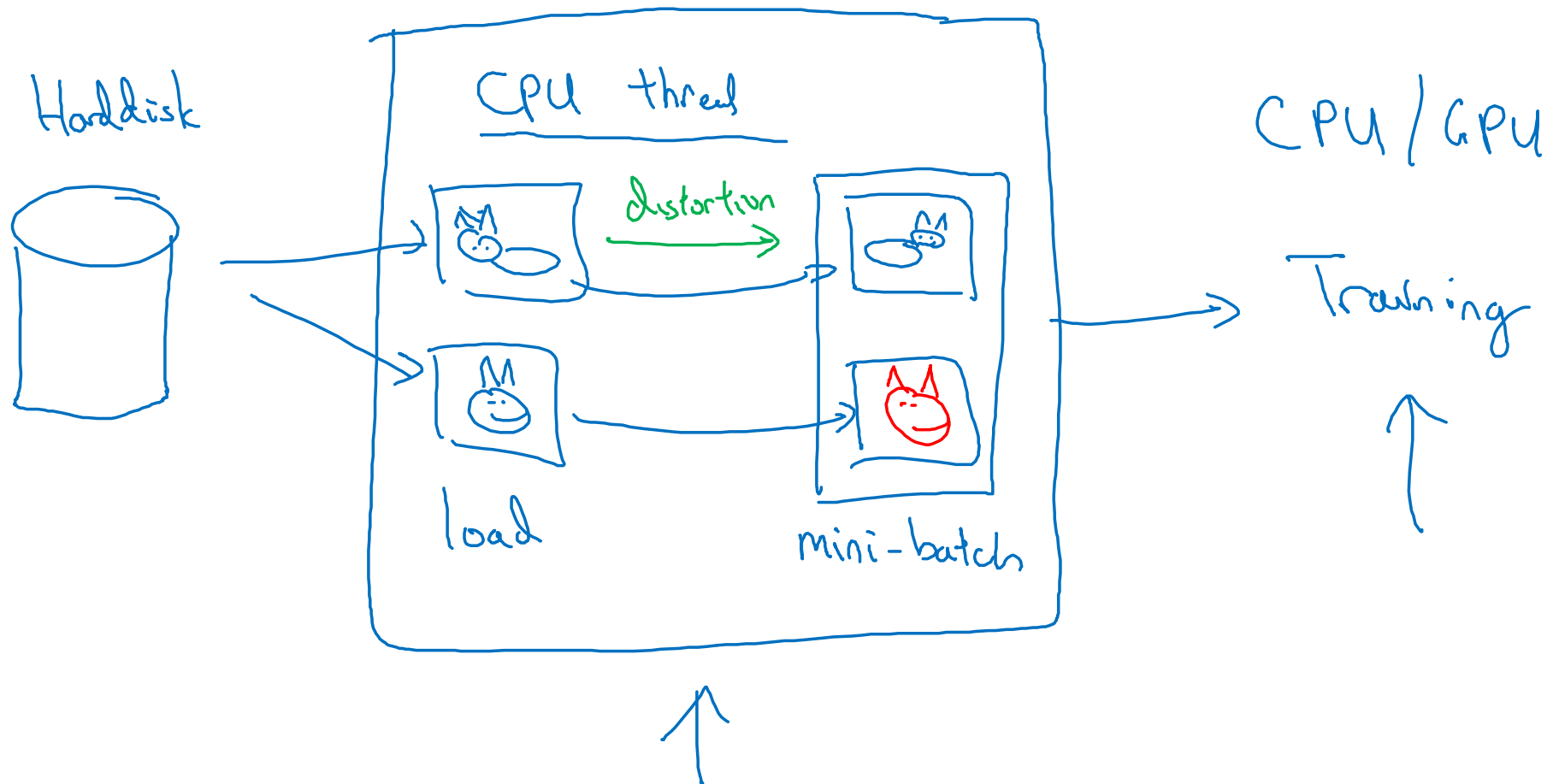
[AlexNet paper

["PCA color augmentation."

R B

G

Implementing distortions during training



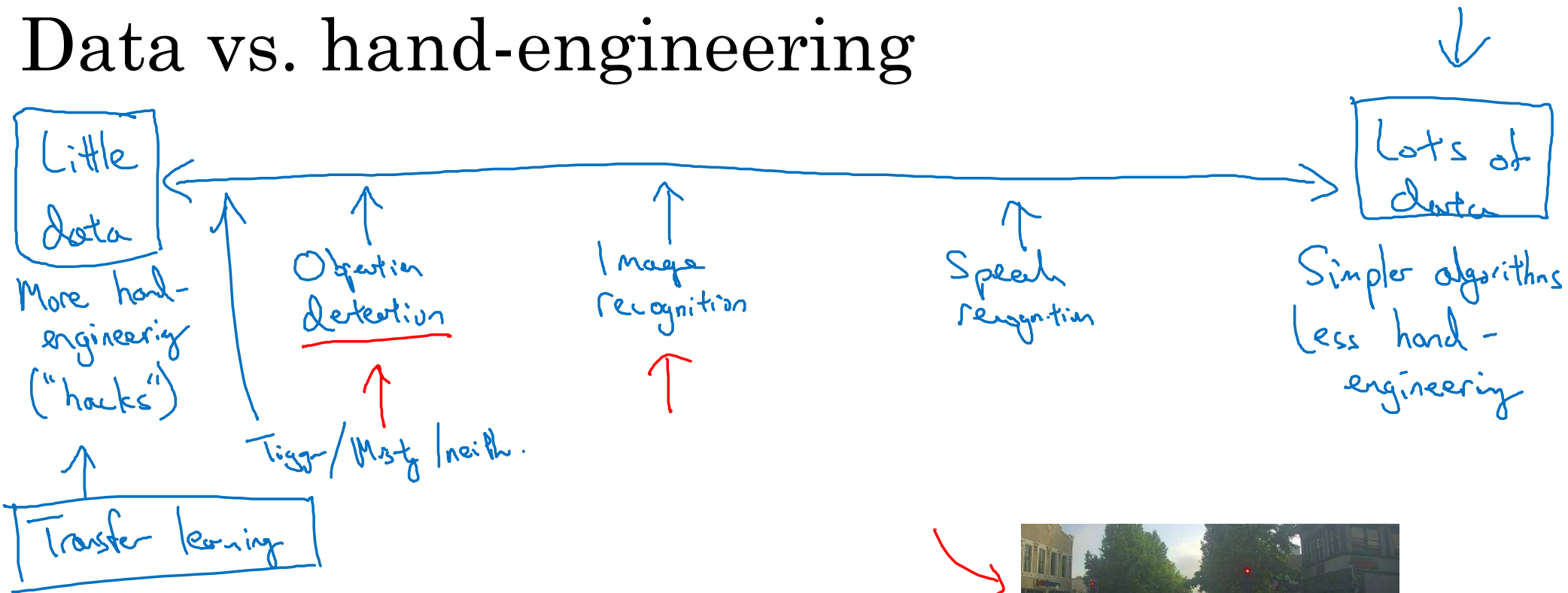


deeplearning.ai

Practical advice for using ConvNets

The state of computer vision

Data vs. hand-engineering



Two sources of knowledge

- • Labeled data (x, y)
- • Hand engineered features/network architecture/other components



Tips for doing well on benchmarks/winning competitions

3-15 networks

→ $\hat{y}$

Ensembling

- Train several networks independently and average their outputs

Multi-crop at test time

- Run classifier on multiple versions of test images and average results

10-crop



1

+



4

+



1

+



4

Andrew Ng

Use open source code

- Use architectures of networks published in the literature
- Use open source implementations if possible
- Use pretrained models and fine-tune on your dataset